

AskTube AI

Chat with YouTube videos through transcript-grounded AI

SEARCH

TRANSCRIPT

RAG

CITATIONS

Netflix-inspired UI + FastAPI + LangChain +
ChromaDB + EC2

Final Project | Sabeur Zarai | IronHack | Docker + EC2

Live demo: <http://18.157.233.122:3001/>

YouTube is huge. Learning from long videos is slow.

AskTube AI turns a video into a searchable, conversational knowledge base.

USER FLOW

Search a topic, choose a video, let the app process the transcript, then ask questions with source timestamps.

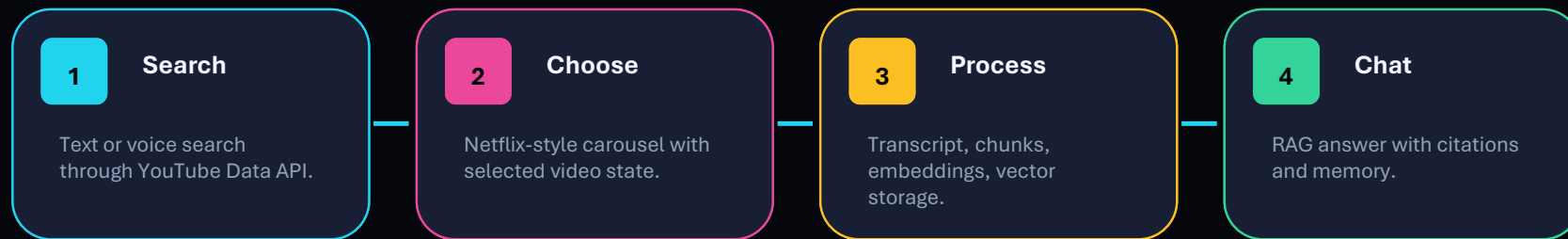
TRUST RULE

The assistant answers from transcript evidence only. If the transcript cannot answer, it refuses instead of inventing.

WHY IT MATTERS

Learners can skip guessing where information lives and jump straight to the exact moment in the video.

From search to cited answer



The UI is intentionally cinematic: dark mode, glass surfaces, duration filters, smooth transitions, a 3D assistant, and graceful loading/error states.

Three layers, one learning loop

FRONTEND

Next.js 14, TypeScript, TailwindCSS, shadcn/ui, Framer Motion, Embla, Three.js, React Three Fiber.

BACKEND API

FastAPI async routes for search, transcripts, chunks, vectorstore, chat, agent, speech, and evaluations.

AI + DATA

LangChain tools and agent, OpenAI models, ChromaDB vector storage, LangSmith tracing, Whisper fallback.

Docker Compose runs frontend, backend, and ChromaDB together for local development and deployment testing.

A YouTube video becomes retrieval-ready text.

- YouTube Data API retrieves metadata, thumbnails, duration, and channel information, then applies the selected length filter.
- youtube-transcript-api is the primary source for public captions and timestamps.
- Whisper is used only as a fallback when captions are unavailable and the deployment can safely fetch audio.
- LangChain chunking keeps timestamp metadata attached to every chunk.
- OpenAI embeddings are stored in ChromaDB for similarity search.

STATUS STREAM

metadata
transcript
chunking
embeddings
vector_storage
ready / error

The processing screen reflects backend progress, not only decoration.

The chatbot uses tools, not just a single prompt.

AgentService binds StructuredTool objects to the LLM and returns answer, citations, toolstens, and session id

search_youtube_videos

extract_transcript

chunk_transcript

store_video_vectors

ingest_video

retrieve_context

answer_question

GUARDRAILS

The final answer must use answer_question, stay transcript-only, and include timestamp citations. Off-topic questions should be refused.

MEMORY

ConversationMemoryService keeps session-based history so follow-up questions remain coherent.

Answers are grounded in transcript evidence.



If the transcript does not contain the answer, the system says it cannot answer from the video. This is the main anti-hallucination behavior.

The interface feels like a learning product, not a backend demo.

VOICE SEARCH

Browser Web Speech API captures live dictation with waveform feedback. If it fails, MediaRecorder sends audio to `/api/speech/transcribe`.

ASSISTANT

A floating 3D robot moves through the journey, speaks short guidance, and opens a compact transcript-grounded chat.

ACCESSIBILITY

Semantic labels, visible focus states, reduced-motion handling, screen-reader status messages, and touch-friendly layouts.

Extra polish: loading skeletons, retry states, TTS answer playback, responsive stacking, and collapsible evidence panels.

The project is testable, explainable, and demo-ready.

98

PYTEST TESTS

17

RAG EVAL CASES

0

BEHAVIORAL FAILS

3

DOCKER SERVICES

- Coverage includes routes, services, tools, WebSocket ingest, speech transcription, and RAG behavior.
- Evaluation cases test answerable questions, refusals, citation accuracy, summaries, memory, and hallucination prevention.
- LangSmith tracing can inspect latency, context, answer quality, and chain behavior.

Demo path: search, prepare, ask, verify.

- Live app: <http://18.157.233.122:3001/> (EC2 deployment with Webshare proxy)
- Search with a duration filter, for example: python tutorial for beginners under 10 minutes.
- Pick a video and click Prepare to show real ingestion progress.
- Ask: What is Python used for?
- Show timestamp citations and the tool breadcrumb.
- Ask a follow-up to demonstrate memory.
- Ask an unrelated question to show refusal behavior.
- Use mic input and Read aloud if time allows.
- Show EC2 health endpoint and explain the YouTube cloud-IP transcript limitation.

WHAT TO EMPHASIZE

This is not a generic chatbot. It is a transcript-grounded learning workflow with tools, memory, vector search, citations, and a polished UI.

Keep the demo focused on trust: every answer links back to the video. EC2 proves deployment; local run proves full transcript access when YouTube blocks cloud IPs.

AskTube AI makes YouTube learnable by conversation.

Search videos. Build transcript knowledge. Ask questions. Verify with timestamps.

MANDATORY REQUIREMENTS

LLM chatbot, tools, memory, vector database, text processing, UI, tests, evaluation, and Docker deployment are covered.

OPTIONAL FEATURES

Voice input, Whisper fallback, WebSocket streaming, TTS, 3D assistant, and LangSmith tracing are included.

NEXT IMPROVEMENTS

Persistent user accounts, multi-video comparison, HTTPS custom domain, and a stronger production transcript proxy.

Thank you